

- Question 1 : Comment écrire la fonction de validation `validateFutureSlot(date, time)` ?

- Réponse (Code) :

JavaScript

```
function validateFutureSlot(dateStr, timeStr) {
  const selectedDate = new Date(`${dateStr}T${timeStr}`);
  if (isNaN(selectedDate.getTime())) return false;
  return selectedDate > new Date();
}
```

- Question 2 : Comment implémenter `proposeAppointment()` pour créer un rendez-vous `proposed` ?

- Réponse (Code) :

JavaScript

```
async function proposeAppointment(patientId, practitionerId, startIso, endIso) {
  const appt = {
    resourceType: 'Appointment',
    status: 'proposed',
    start: startIso,
    end: endIso,
    participant: buildParticipants(patientId, practitionerId, 'needs-action', 'needs-action')
  };
  return await fhirFetch('/Appointment', {
    method: 'POST',
    body: JSON.stringify(appt)
  });
}
```

- Question 3 : Comment structurer le tableau obligatoire `participant` selon la cardinalité 1.. FHIR R4 ?*

- Réponse (Code) :

JavaScript

```
function buildParticipants(patientId, practitionerId, patientStatus, practStatus) {
  return [
    {
      actor: { reference: `Patient/${patientId}` },
      status: patientStatus
    },
    {

```

```

    actor: { reference: `Practitioner/${practitionerId}` },
    status: practStatus
  }
];
}

```

- Question 4 : Comment concevoir la fonction générique `updateAppointment(id, patchData)` via `PUT` ?

- Réponse (Code) :

JavaScript

```

async function updateAppointment(appointmentId, updatedResource) {
  return await fhirFetch(`/Appointment/${appointmentId}`, {
    method: 'PUT',
    body: JSON.stringify(updatedResource)
  });
}

```

- Question 5 : Comment implémenter `acceptAppointment()` pour valider le RDV au statut `booked` ?

- Réponse (Code) :

JavaScript

```

async function acceptAppointment(currentAppt) {
  const updated = {
    ...currentAppt,
    status: 'booked',
    participant: currentAppt.participant.map(p => {
      if (p.actor.reference.startsWith('Patient/')) {
        return { ...p, status: 'accepted' };
      }
    })
  };
  return await updateAppointment(currentAppt.id, updated);
}

```

- Question 6 : Comment coder `declineAppointment()` pour annuler le rendez-vous (`cancelled`) ?

- Réponse (Code) :

JavaScript

```

async function declineAppointment(currentAppt) {
  const updated = {
    ...currentAppt,
    status: 'cancelled',
    participant: currentAppt.participant.map(p => {
      if (p.actor.reference.startsWith('Patient/')) {
        return { ...p, status: 'declined' };
      }
      return p;
    })
  };
  return await updateAppointment(currentAppt.id, updated);
}

```

- Question 7 : Comment écrire `sendReminder()` qui génère et poste une ressource `Communication` ?

- Réponse (Code) :

JavaScript

```

async function sendReminder(patientId, messageText) {
  const comm = {
    resourceType: 'Communication',
    status: 'completed',
    subject: { reference: `Patient/${patientId}` },
    payload: [{ contentString: messageText }],
    sent: new Date().toISOString()
  };
  return await fhirFetch('/Communication', {
    method: 'POST',
    body: JSON.stringify(comm)
  });
}

```